

Technical Plan: Data-Driven Generative Visualizer (4-Bit Grayscale)

System Overview & Goals

This project will be implemented as a **modular software system** that generates real-time visuals in **16-level grayscale**. The system will be **data-driven** (influenced by external sensor or Home Assistant data) but designed such that the **core logic and aesthetic rules are clear and adjustable** by the user. In the initial phase, we'll use **simulated inputs** (random or scripted data feeds) to develop and test the logic. This ensures we understand how the algorithms work **before** hooking up real Home Assistant data.

Key Objectives Recap: The system should produce compelling grayscale visuals with limited tonal resolution, be tunable by non-experts, and reflect external data in a legible yet artistic way. We will maintain a one-way flow of influence: **data → visuals** (no feedback from visuals back to sensors).

Architecture Design

To manage complexity, the software will be organized into **three main layers**, each with a distinct responsibility. This layered design echoes approaches used by generative art pioneers, who emphasized clear **rules and systems** in their work ¹ ² :

- **Input Layer:** Handles incoming data (sensor readings, HA states, etc.) and provides a **current data snapshot** to the rest of the system. In early development, this layer will output *simulated or randomized data streams* instead of real sensor data (e.g. using random number generators or sine wave functions to mimic changing values). This allows us to prototype the visual logic without needing a live sensor hookup. The input layer will eventually be replaced or augmented by a **Home Assistant data adapter** (e.g. an MQTT or REST client) once the visualization logic is solid.
- **Mapping Layer:** Transforms raw input values into **normalized parameters** and high-level **semantic channels**. This is where we interpret sensor data in ways that affect the visuals. The mapping layer will combine and scale inputs into meaningful variables – for example:
 - `activity_level` could be a blend of motion sensor counts and room occupancy data.
 - `energy_intensity` might derive from home energy usage (Watts) normalized to a 0–1 range.
 - `climate_variability` could combine temperature and humidity readings (or their rate of change).

Each of these channels ranges over a fixed interval (say 0.0 to 1.0) and has a defined influence on the visual output. By separating this layer, we make it easy to adjust how data maps to visuals without changing the drawing code. **Weighting configurations** live here: e.g. *"density = 0.7 * energy + 0.3 * humidity"* as a formula. During prototyping, these weights can be **chosen arbitrarily or even randomized** to experiment with different influences (as you suggested, we might start with random weights each run to see a variety of

behaviors). Over time, we'll tune them to achieve mappings that are both *legible* and *aesthetically pleasing*. The goal is to avoid straightforward graph-like representations and instead find **subtle or metaphorical mappings**. For instance, rather than directly drawing a bar for temperature, a high temperature might gradually increase the turbulence in a particle system – a less literal, more artistic reflection of “heat.”

- **Visual Engine Layer:** This layer is responsible for the **generative graphics output**. It consists of one or more **visual modules** (algorithms) that read the parameters from the mapping layer and produce drawings/animations accordingly. Each module encapsulates a particular aesthetic style or technique (for example, a grid of shifting shapes, a field of particles, a flowing noise texture, etc.). The modules share a common interface so they can be swapped in and out:
- Each module will implement an `update(params)` function that takes the current parameter set (from the mapping layer) and uses it to update the module's state (e.g. move points, add shapes, change speeds).
- Each module also implements a `draw(canvas)` (or simply draws to the screen) function that renders the current state to the display in 4-bit grayscale.

By enforcing a minimal interface (much like a modular synthesizer where each module has input knobs and an output signal), we ensure new visual algorithms can be added without modifying the rest of the system. This draws inspiration from systems like Casey Reas's **Process** series, where the *rules* (analogous to our module code) are separate from the specific images they produce ¹. Here, our “rules” are the code in each module that dictates how visuals evolve in response to input parameters.

Output Format & Grayscale Constraint: All drawing will be constrained to 16 discrete grayscale levels (0–15). To achieve this: - We will quantize any continuous tone output into 16 steps. If using a graphics library that outputs 8-bit grayscale (0–255), we map those values to the nearest 4-bit level (e.g., $\text{value} = \text{floor}(\text{pixel_value} / 16)$). - To create the illusion of smoother gradients or richer texture within those 16 shades, we can employ **dithering techniques**. For instance, an **ordered dithering matrix** or **Floyd-Steinberg error diffusion** could be applied when rendering images with gradients. Dithering will disperse quantization error in a pattern, tricking the eye into seeing intermediate tones by the arrangement of dark and light pixels. This is optional for certain styles – in some cases a stark 16-tone image (like a posterized look) might be aesthetically desired. We'll decide per module whether to use dithering or not, to balance clarity versus visual richness. - We will leverage **contrast, shape, and motion** heavily, since we cannot rely on color. Techniques from early computer art and printmaking (strong silhouettes, hatching or stippling for mid-tones, moiré patterns, etc.) can be useful. For example, **Vera Molnár** achieved remarkable variation using just lines and simple shapes with slight rule-bending ³. We can similarly introduce small procedural randomness (Molnár's famous “1% disorder”) into our drawing routines to avoid overly rigid output and give it a humanized, lively feel ³.

Ensuring One-Way Data Flow

Architecturally, the system will **not send any data or commands back** to the Home Assistant/sensor side. The Input Layer client will be strictly read-only. This guarantees our visuals do not **influence the environment**, they only reflect it. This one-way design simplifies the logic (no feedback loops to manage) and aligns with the concept of the system as an **ambient mirror** of the home's state. In code, this just means we never call any HA service or write to any actuator – only subscribe/read. It keeps the relationship conceptually clear and avoids any unintended side-effects in the home when experimenting with the visuals.

Development Plan & Phases

We will implement the project in incremental phases, validating the logic at each step:

Phase 1 – Prototype Visuals with Simulated Data (on PC/Mac): We begin on a powerful machine (your Mac mini) for convenience and speed. In this phase, we'll write several simple generative drawing sketches *without real data*. Instead, we'll create dummy data streams: e.g., a slowly oscillating value to mimic temperature, a random spike generator to mimic motion events, etc. The goal here is to exercise the **Mapping and Visual layers** in isolation. We'll develop a rudimentary **data simulator** module that produces a JSON or dictionary of key→value pairs (one for each data channel, like `temperature: 0.65`, `occupancy: 0.2`, etc.), updating them over time. This simulator can have modes (random, periodic, or even playback of recorded data if available).

During this phase, we also establish the **software framework** and libraries: - Decide on the programming environment. Two strong options are: - **Python** (with libraries like Pygame or PyOpenGL for rendering) which is quick to develop and has good library support, or - **C++ with openFrameworks** for potentially better performance and graphics capabilities on the Pi ⁴. - Given the Raspberry Pi target, using OpenGL ES or SDL via C++ might yield smoother performance for graphics-heavy sketches. **openFrameworks** (C++ toolkit) is attractive here because it's built for creative coding and has cross-platform support (Linux/ARM for Pi, and macOS) ⁴. Many generative artists (like Zach Lieberman) use openFrameworks to get efficient real-time graphics on limited hardware ⁴. On the other hand, Python might be easier to integrate with Home Assistant's Pythonic ecosystem. We might start with Python for simplicity and only move to C++/openFrameworks if performance tests in Phase 3 demand it. - We create the skeleton code for **modules**. For example, if using Python, we can define an abstract base class `VisualModule` with methods `update(params)` and `draw(surface)`. Then each specific visual effect will subclass this. If using openFrameworks/C++, we might define a base class or simply adhere to a pattern (since C++ doesn't have a built-in interface concept like Java, but we can use abstract classes or function pointers). - Implement one simple visual module first – for instance, a grid of rectangles whose sizes shift based on a parameter. This will test end-to-end flow: simulator → mapping → one visual output. We'll verify that adjusting the simulated input indeed changes the graphics as expected (ensuring our parameter mapping is doing something perceptible).

Phase 2 – Data Pipeline Integration: With a working prototype of the visual system, we'll integrate real data from the external source: - Set up a connection to **Home Assistant**. This could be through HA's **WebSocket API** or **MQTT** if HA is configured to publish sensor states. Another option is periodic REST calls to HA's API to fetch sensor values. MQTT is great for near-real-time streaming of sensor updates with low overhead. - Implement the **Input Layer client** that subscribes to relevant data topics (e.g., temperature, humidity, motion, power usage topics). It will receive updates and maintain a global state dictionary of the latest values for each channel. - Feed this real data into the Mapping layer. We'll need to normalize and clamp values. For example, if temperature comes in as 72°F, the mapping layer might subtract a baseline and divide by a range to yield a 0–1 value. Where possible, use **moving averages or smoothing filters** on raw data to prevent overly erratic visuals (unless that's desired for something like instantaneous motion spikes). - At this stage, the visual modules should start responding to actual home events. We might run the system on the Mac (still) but pulling data from the Pi or HA over the network, just to test the whole loop in a comfortable dev environment.

We will likely discover that some data channels are noisy or not very dynamic, which could lead to visuals that are either jittery or too static. We should refine the mapping functions here: - Possibly introduce thresholds or nonlinear mapping. E.g., use a logarithmic scale for power usage so that moderate changes are visible but a huge spike doesn't completely overwhelm the parameter. - Ensure each data stream has a **meaningful effect** on the image. We don't want a parameter that never changes or one that changes so fast it just looks random. Tuning and observing in this phase is crucial.

Phase 3 – Port to Raspberry Pi & Optimization: Now we transfer the system to the Raspberry Pi hardware (our target small device). Key tasks and considerations: - **Deploy environment:** Install necessary libraries on the Pi. If using Python, that means Pygame/PyOpenGL, Numpy, etc. If using openFrameworks C++, we will compile our code for ARM (openFrameworks has build configs for Raspberry Pi). We need to test that the Pi's GPU (if any, e.g. using OpenGL ES) or CPU can handle our rendering at a decent frame rate. - **Performance tuning:** Likely we'll need to adjust parameters for the Pi. This could include: - Lowering the output **resolution** or update **frame rate** if needed. For example, rendering at 720p instead of 1080p to lighten the load, or updating visuals 20 times per second instead of 60 if full frame rate isn't achievable, given that some generative art looks fine at lower fps or can interpolate. - Simplifying visuals that are too computationally heavy. If a module uses hundreds of objects and the Pi struggles, we might cap the number of elements or find more efficient drawing methods (like using PyOpenGL to draw many points via a single shader, or in openFrameworks using VBOs). - Utilizing the Pi's strengths: If on a Pi 4 with the VideoCore GPU, we might leverage hardware-accelerated paths. OpenFrameworks automatically uses hardware acceleration for drawing shapes, but in Python we might consider using OpenGL shaders for expensive effects. - **Testing on target display:** If the final installation uses a specific display (say a small e-ink screen or just an HDMI monitor in grayscale mode), test the grayscale output. Ensure that our 16 shades are distinguishable on that display. Some displays might have gamma issues – e.g., the darkest few grays might all look black. We may need to tweak the mapping to the 0–15 range (maybe use a gamma curve or adjust specific levels). - Confirm that the one-way data feed works on the Pi. The Pi might itself run Home Assistant or be networked to the main HA server. In either case, ensure our Input Layer client on the Pi is receiving data timely. We'll run the visualizer for extended periods to see stability (memory leaks, network hiccups, etc.). Using a Raspberry Pi also invites the possibility of attaching **direct sensors** (like a Pi temperature sensor GPIO) in the future, but for now, HA data is the assumed source.

Phase 4 – User Controls & Modularity Enhancements: Once the core system is running smoothly on the Pi, we'll improve the **adjustability and user interface**: - Implement a simple **configuration file** or **web-based UI** that exposes the main adjustable parameters. For instance, a JSON or YAML config could list the current weightings and module settings. A small web server on the Pi (possibly a lightweight Flask app or even just an HTTP endpoint if using openFrameworks with ofxAddon for web UI) could present sliders and buttons. This UI doesn't have to be fancy; even controlling via SSH or a text file that the program reads is okay in early stages. The important part is that a **non-programmer user** can tweak things like “weight of temperature on visual blur” or switch from Module A to Module B. - Introduce the concept of **presets or scenes**: a saved set of parameter mappings and chosen module. The user can load a preset to quickly change the overall behavior (e.g. a “Calm mode” preset vs. an “Energetic mode”). We might implement this as multiple config files or a menu in the UI. - Expand the library of **visual modules**. We started with one or two in earlier phases; now we can add more. Thanks to our modular architecture, this is easier – each new module just needs to conform to the `update/draw` interface and can reuse the existing data feed. Examples of modules we can attempt: - A **Particle System**: many small circles or dots that wander. Data mappings: “activity_level” controls the number of particles and their speed; “energy_intensity” could control the direction bias or size of particles (e.g., high energy = particles trace longer streaks or move more

erratically). - A **Grid of Lights**: think of a grid of 2D cells (like an 16×16 matrix) that light up or change shade. Data could influence patterns like Conway's Game of Life step rate, or simply the noise that turns cells on/off. For instance, "occupancy" might increase the density of lit cells, whereas "climate_variability" (temp/humidity changes) might introduce flicker or turbulence in the pattern. - **Waveform or Line-based Drawing**: e.g., a set of horizontal lines that undulate like an abstract horizon. Data mapping: one channel could modulate the amplitude of waves, another the frequency of the wiggles, another the overall slope or drift of the lines. This could create a visual metaphor of "calm vs chaotic" environment – calm data yields gently waving lines, busy data yields jagged oscillations. - **Typographic/ASCII Art Forms**: Since 4-bit grayscale evokes old terminals or e-ink, one module could use characters or blocks that fill the screen, whose darkness corresponds to a data channel. For example, if you have a set of 16 ASCII characters from light to dark (like `. : -=+*#%@` etc.), the program could fill the screen with a character whose index is proportional to, say, current energy use (higher energy → heavier character). This is a more playful, less abstract module.

Each new module we add will test the generality of our system. We might refine the parameter set or add new ones if a visual style needs something specific (for example, a "color temperature" parameter is irrelevant in grayscale, but a "pattern randomness" parameter might be needed for a certain algorithm). We should strive to keep the parameter set **consistent** across modules for the user's sake – e.g., always use `activity_level` to mean something akin to motion or busyness, even if module A uses it for particle count and module B uses it for line turbulence. Consistency will make it easier for a user to predict what increasing a slider will do in different visuals.

- **User Testing and Iteration**: At this stage, if possible, involve a few test users (or ourselves taking on a user perspective after a break) to see if the controls and outputs are understandable. Does the user roughly grasp "ah, this looks more intense when more people are home" or "this pattern calms down when it's quiet at night"? If not, we might need to adjust mappings or add on-screen indicators (perhaps subtle text overlays or LED indicators in a physical installation) to guide interpretation. However, we should avoid anything too literally informational – the display is ambient art, not a dashboard.

Throughout all phases, maintain documentation: we'll keep notes on what each parameter does, how each module works, and any decisions (like why we scaled X this way or chose Y algorithm). This will help in writing the final report/reflection.

Mapping Strategies & Algorithm Design

Designing the mapping from data to visuals is the crux of making the system **legible yet intriguing**. Here we outline how we'll craft the logic and some example algorithms:

- **Data Normalization**: All sensor inputs will be normalized to a common scale. We'll define for each data type a plausible min-max range (either based on real physical limits or historical data). For example, temperature might map 60°F to 0.0 and 80°F to 1.0 (if the home is always between those), whereas something like "number of people detected" might map 0 people to 0.0 and 5 people to 1.0 (assuming 5 is a full house). Values outside the expected range can be clipped or could be mapped in a nonlinear way if needed (to avoid saturation effects).

- **Weight Combinations vs. Direct Mappings:** In some cases, one sensor directly drives one visual parameter (e.g., **time of day** from HA could directly set a background brightness – darker at night). In other cases, a mix of inputs is more interesting. We will use **weighted sums** or functions to combine inputs into “meta-parameters.” For instance, an “**activity_level**” parameter might be computed as:

```
activity_level = 0.5 * normalized(motion_sensor_count) + 0.5 *
normalized(noise_level)
```

If such data exists. Early on, if we aren’t sure, we can indeed assign **random weights** or try extremes (all weight on one input) to subjectively judge what produces the most engaging result. Over time, we’ll refine these by observing the output when feeding in recorded real data. The mapping logic will likely live in a function that reads the raw data dict and outputs a params dict. This function can be made easily configurable (perhaps reading a config file of weights, or code constants at first).

- **Temporal Smoothing & Change Detection:** A critical part of the algorithm is how to handle time. If data changes too rapidly, the visual may flicker or become noise. If it’s too slow, the visual might be boring. We can incorporate logic like:

- Running averages or low-pass filters on inputs for smoother changes.
- Alternatively, detect **trends or sudden changes** and map those to distinct visual responses. E.g., if energy usage jumps quickly (maybe someone turned on an appliance), that *change* could trigger a one-time burst in the visuals (like a flash or a momentary pattern) separate from the continuous state. This adds another layer of dynamism: not just reflecting the steady-state values but also the first derivatives (deltas). We can incorporate simple state memory in the mapping layer to compare current vs previous values.
- Time-of-day or cyclical behaviors: Perhaps introduce subtle shifts over the day even if data is constant, to keep visuals from freezing when environment is steady. For example, a slow oscillation could be applied to a parameter so that even if all sensors are static, the visual still breathes a little (preventing burn-in on displays and adding a sense of life).

- **Examples of Data-to-Visual Mappings (Logic):** To ground this, consider a specific **Visual Module** and walk through how data influences it:

- *Example Module: “Flow Field”:* Imagine a field of short line segments that resemble a flow or wind pattern. They could be drawn across the screen, each oriented in some direction. How might data drive this? We could say:

- The **wind direction** displayed is driven by an average of outside wind direction (if available from weather API) or simply a fixed noise pattern (if no direct analog).
- The **turbulence or variability** in the flow is tied to, say, the number of people at home or recent motion. When more activity is detected, the flow lines get more chaotic or jittery. This could be done by increasing a noise field’s amplitude or the randomness of each line’s angle.
- The **speed** of movement of these lines (if they slowly drift) could correlate with energy usage – high power usage speeds everything up (as if the system is “humming” with electricity).
- Now, implementing this: in `update(params)`, we might have

```
for each line: angle = base_flow(x,y) + randomness * activity_level;
length = base_length; drift_speed = 0.5 + energy_intensity * 1.0
```

, etc. So

the code clearly ties `activity_level` to the amount of random perturbation, and `energy_intensity` to how fast lines move.

- In `draw()`, we render each line with a grayscale determined maybe by another parameter (e.g., perhaps **temperature** controls brightness: hotter = brighter lines). This way, multiple data points layer into one image in different facets (motion affects shape, energy affects motion, temperature affects brightness).
 - Because this could get confusing, we'd document these mappings in the code and perhaps in the user guide, so it's clear ("In Flow Field module: occupancy influences turbulence, energy influences flow speed, temperature sets base gray level" etc.).
- **Example Module: "City Lights":** Think of a stylized city skyline or bar graph that grows and shrinks. This could be represented by a row of vertical bars (or building silhouettes). Data mapping:
 - **Each bar** could correspond to a different sensor (e.g., different rooms' motion or different appliance power usage). Then the height of the bar is the current value. This is a more literal mapping, almost infographic, but we could make it abstract by not labeling them and making the bars semi-random widths.
 - Alternatively, treat the whole set of bars as one shape and use a single parameter to control an overall pattern – for example, "activity_level" could control the jaggedness or roughness of the skyline top (higher activity makes the skyline flicker with tiny highs and lows, low activity makes it a smooth gentle curve).
 - The grayscale mapping could be inverted here: maybe one parameter (like time of day or a recent change) causes the "lights" to turn on/off (white vs black bars). For example, at night perhaps the background goes dark and bars are lighter (like lit windows), whereas in day the background is light and bars dark. This introduces an environmental context (time) as a data input too (from HA's clock or sun state).
 - The logic for this module might be simpler math – directly mapping values to heights and colors. It's important we still apply aesthetic considerations (ease the transitions, maybe quantize heights to some grid so they don't look like raw analog bars, etc., to keep an artistic feel).
 - **Algorithm Inspiration from Art History:** We can take cues from the references:
 - **Manfred Mohr's fixed geometric structures:** Mohr built works by systematically iterating through combinations of a base form (like edges of a cube) ². We can mirror this by picking a geometric motif (say a cube wireframe, a grid, a spiral) and then use data to pick which portions to draw or how to transform it. For instance, imagine a wireframe cube where sensor data decides the rotation angle or which edges are highlighted. The logic would involve predefining the cube's coordinates and then using data values to rotate it or fade certain edges in/out. While this might be more static than others, it could serve as a calm, structured visualization – and it resonates with the idea of exploring combinations under constraints ².
 - **Vera Molnár's subtle randomness:** As noted, we can incorporate tiny randomness to avoid perfectly repetitive output ³. Algorithmically, this means any time we have a repetitive structure (like a grid of identical shapes), we add a small random offset that's constant for a while (to not flicker per frame, we can choose a random offset per shape on initialization). This makes the output look less machine-perfect and more organic, improving its expressive quality without losing the underlying pattern.

- **Casey Reas's process of emergent complexity from simple rules:** We might design one module to be an **agent-based simulation** (like boids or cellular automata), which inherently can produce complex emergent visuals. The logic is: define simple behaviors for elements and let them interact. We can let data influence parameters of those behaviors (e.g., a Conway's Life where the birth/death rules or probabilities are skewed by a sensor). The result won't directly encode the data in a graph, but the *character* of the evolving pattern will shift. This echoes Reas's idea of software as generative rule systems rather than static compositions ¹. We will need to carefully observe if such systems still reflect the data changes in a noticeable way or if they drown in their own complexity. If the latter, we scale back complexity for clarity.

Tools, Libraries, and Hardware Considerations

- **Graphics Library:** As mentioned, we will choose between Python and openFrameworks C++. For concreteness, suppose we start with **Python** to quickly prototype (using Pygame for a window and drawing shapes, or possibly **Processing's Python mode** for convenience if preferred). Python allows fast tweaking, and there are image libraries (PIL/Pillow) if we need to do pixel-by-pixel operations (like custom dithering). If performance lags, we can optimize hotspots (e.g., using NumPy to vectorize computations, or writing a C extension for heavy math).
- **Raspberry Pi specifics:** We'll likely use a Raspberry Pi 4 (since it was mentioned as a target). The Pi 4 can run an X server or framebuffer for graphics. If performance is an issue, we might render at a smaller off-screen surface and then scale it up. Since the output is just grayscale, even dropping to a lower resolution and relying on the display's own scaling might not look bad (could even add to a pixel-art aesthetic). Also, ensure the Pi has GPU drivers set up for OpenGL ES if using hardware accel.
- **Home Assistant integration:** If Home Assistant is running on the Pi itself, we might have direct access to sensor data via local files or databases. More commonly, we'll connect over the network. We should handle potential disconnects gracefully – e.g., if HA goes down or Wi-Fi drops, the visualizer should not crash. We can implement reconnection logic or at least hold the last values and perhaps slowly fade to an idle pattern if data is stale for too long (like if no new data for, say, 5 minutes, maybe the visuals enter a standby or loop on a default calm animation).
- **Persistence:** The system's adjustable settings (weights, chosen module, etc.) will be saved in a config (possibly JSON) so that if the Pi reboots or the program restarts, it can recall the last setup. This is straightforward to implement by writing the config whenever changes are made in the UI or by loading it on start.
- **Testing & Iteration speed:** We will maintain the ability to run the program on a PC where possible to leverage debugging and faster dev cycle. We could simulate the Pi environment or use a subset of data. openFrameworks code can be compiled and run on Mac/Windows as well, which is handy. If using Python, we just need to ensure any Pi-specific libraries are abstracted or not required on PC (for example, if using a library to read GPIO on Pi, on PC we'd stub that out).

Evaluation Plans (Post-Implementation)

(This is more forward-looking, but it's important to define how we'll know the system is successful and identify areas to refine.)

Once the system is up and running with actual home data, we will evaluate it along several dimensions relevant to our design goals:

- **Data Legibility:** Watch the real-time output and note if changes in the home are observable in the visuals. For instance, does the morning routine (lights turning on, coffee maker power spike, people waking up) correspond to a noticeable shift in the display's behavior? If a viewer knows the general metaphor (e.g., "this wavy pattern = activity levels"), they should at least detect *relative* changes ("it's more chaotic now than before, probably something happened"). We might even do a simple test: deliberately trigger a change (turn on a device, simulate occupancy) and see if the visual responds in the expected manner.
- **Aesthetic Engagement:** Gather feedback on the *look and feel*. With the 4-bit grayscale constraint, does the output still appear rich and interesting? We'll pay attention to whether the lack of color feels like a strength (cohesive, classic look) or a limitation (e.g., some data influence might be clearer if color was used). If the latter, we might need to compensate by adjusting brightness or contrast mappings. We will also check the effect of dithering – is it improving the perceived quality or is it distracting? Fine-tuning the dithering pattern (matrix size, error diffusion strength) might be needed per module.
- **Modularity & Extensibility:** Try adding a totally new visual module late in the project and see how smoothly it integrates. If it requires a lot of special-case code, our architecture might need refactoring. Ideally, a new module should be pluggable by just registering it in a list of modules and hooking up its parameters. This is analogous to how audio software plugins work and is a good test of modular design.
- **User Tuning Experience:** If we have a UI for controls, let a friend or colleague use it without much guidance. See if the labels and ranges make sense. For example, if a slider is labeled "Energy→Density weight", does adjusting it have a noticeable and understandable effect on the visuals? If not, maybe the control scheme needs simplification (perhaps group multiple low-level weights into one "mood" slider, etc.). We want users to feel they can **explore** the system creatively, not get lost in technical minutiae.
- **Performance & Stability:** Finally, ensure the Raspberry Pi can run the system continuously (e.g., 24/7 if intended as an ambient display). Monitor CPU/GPU usage and temperature. If the Pi is near its limits, we might underclock visuals when idle (e.g. at night if nobody is around, maybe automatically reduce frame rate to let the Pi rest, unless the night pattern is itself essential). We should also confirm that memory usage doesn't grow (no memory leaks in our loops).

By iterating on these evaluation points, we will refine both the **logic** (mapping algorithms, parameter choices) and the **presentation** (visual algorithms, interface) of the system.

Conclusion

This technical plan lays out how to proceed from concept to a working prototype of the grayscale generative art system. By **starting with simulated data and a clear architecture**, we ensure that we understand the logic of how data influences art before dealing with real-world complexities. The approach is informed by the practices of generative art pioneers – using well-defined rules and embracing constraints to spur creativity (as seen in Mohr's algorithmic approach ² and Molnár's balance of order/chaos ³). We also take inspiration from creative coding frameworks like Processing and openFrameworks that champion accessibility and modularity in art-making software ¹ ⁴.

By the end of implementation, we expect to have a **versatile system** where swapping data inputs or visual styles is straightforward. The logic will be transparent enough that tuning the behavior doesn't require diving into code – fulfilling the goal of being *understandable and adjustable by non-experts*. And most importantly, the final visuals should provide an **ambient but meaningful reflection of the home environment**: a piece of “living art” that quietly mirrors life's rhythms in 4-bit grayscale glory.

1 Process — REAS.com

<https://reas.com/process>

2 Cubic Limit by Manfred Mohr - bitforms gallery

<https://www.bitforms.art/artwork/cubic-limit-3>

3 4 Generative Art with Zach Lieberman

<https://baukunst.co/stories/collabs/generative-art-with-zach-lieberman>